```
import time
import struct
from machine import Pin, PWM, I2C, UART

# ——————————————————————————————
#  UART1 → ESP8266 LoRa TX
# ——————————————————————————————
uart_lora = UART(1, baudrate=115200, tx=Pin(4), rx=Pin(5))

# ——————————————————————————————
#  MOTOR PINS & SPEED CONFIG
# ——————————————————————————————
L_IN1 = Pin(6, Pin.OUT); L_IN2 = Pin(7, Pin.OUT)
L_ENA = PWM(Pin(8));    L_ENA.freq(1000)

R_IN3 = Pin(9, Pin.OUT); R_IN4 = Pin(10, Pin.OUT)
R_ENB = PWM(Pin(11));   R_ENB.freq(1000)

SPEED_REGULAR = int(65535 * 0.455)  # ~45.5% throttle
SPEED_SLOW    = int(65535 * 0.40)   # ~40%   throttle

# ——————————————————————————————
#  DISTANCE THRESHOLDS
# ——————————————————————————————
DIST_SLOW     = 20  # cm – start braking
DIST_STOP     = 7   # cm – evade
DIST_CRITICAL = 4   # cm – emergency reverse & scan
```

```
# ──────────────────────────────────────────────
#  ULTRASONIC SENSORS (3-array)
# ──────────────────────────────────────────────
UL_TRIG = Pin(18, Pin.OUT); UL_ECHO = Pin(19, Pin.IN)
UC_TRIG = Pin(20, Pin.OUT); UC_ECHO = Pin(21, Pin.IN)
UR_TRIG = Pin(14, Pin.OUT); UR_ECHO = Pin(15, Pin.IN)

# ──────────────────────────────────────────────
#  IMU (MPU-6050)
# ──────────────────────────────────────────────
i2c     = I2C(0, sda=Pin(0), scl=Pin(1), freq=400_000)
MPU_ADDR = 0x68
i2c.writeto_mem(MPU_ADDR, 0x6B, bytes([0x00]))   # wake up
time.sleep_ms(100)
i2c.writeto_mem(MPU_ADDR, 0x1B, bytes([0x00]))   # gyro ±250 °/s

# ──────────────────────────────────────────────
#  TELEMETRY CONFIG
# ──────────────────────────────────────────────
ROVER_ID       = "R2"
MPU_TX_INTERVAL = 500   # ms between MPU telemetry packets
_last_mpu_tx    = 0
_current_motion = "STOP"


# ═══════════════════════════════════════════════════════════
#  MOTOR CONTROL
# ═══════════════════════════════════════════════════════════
```

```
def stop():
    global _current_motion
    _current_motion = "STOP"
    L_IN1.value(0); L_IN2.value(0); L_ENA.duty_u16(0)
    R_IN3.value(0); R_IN4.value(0); R_ENB.duty_u16(0)

def forward(speed):
    global _current_motion
    _current_motion = "FWD"
    L_IN1.value(1); L_IN2.value(0); L_ENA.duty_u16(speed)
    R_IN3.value(1); R_IN4.value(0); R_ENB.duty_u16(speed)

def backward(speed):
    global _current_motion
    _current_motion = "REV"
    L_IN1.value(0); L_IN2.value(1); L_ENA.duty_u16(speed)
    R_IN3.value(0); R_IN4.value(1); R_ENB.duty_u16(speed)

def turn_left():
    global _current_motion
    _current_motion = "TURN_L"
    L_IN1.value(0); L_IN2.value(1); L_ENA.duty_u16(SPEED_REGULAR)
    R_IN3.value(1); R_IN4.value(0); R_ENB.duty_u16(SPEED_REGULAR)

def turn_right():
    global _current_motion
    _current_motion = "TURN_R"
    L_IN1.value(1); L_IN2.value(0); L_ENA.duty_u16(SPEED_REGULAR)
```

```
    R_IN3.value(0); R_IN4.value(1); R_ENB.duty_u16(SPEED_REGULAR)

def set_turn_motors(direction):
    global _current_motion
    if direction == "left":
        _current_motion = "TURN_L"
        L_IN1.value(0); L_IN2.value(1)
        R_IN3.value(1); R_IN4.value(0)
    else:
        _current_motion = "TURN_R"
        L_IN1.value(1); L_IN2.value(0)
        R_IN3.value(0); R_IN4.value(1)


# ═══════════════════════════════════════════════════════
#  SENSOR READINGS
# ═══════════════════════════════════════════════════════
def get_distance(trig, echo):
    trig.value(0); time.sleep_us(2)
    trig.value(1); time.sleep_us(10)
    trig.value(0)
    timeout = time.ticks_us()
    while echo.value() == 0:
        if time.ticks_diff(time.ticks_us(), timeout) > 20000: return 999
    start = time.ticks_us()
    while echo.value() == 1:
        if time.ticks_diff(time.ticks_us(), start) > 20000: return 999
    return (time.ticks_diff(time.ticks_us(), start) * 0.0343) / 2
```

```
def read_accel():
    raw  = i2c.readfrom_mem(MPU_ADDR, 0x3B, 6)
    vals = struct.unpack('>3h', raw)
    return [v / 16384.0 for v in vals]

def read_gyro_z():
    raw = i2c.readfrom_mem(MPU_ADDR, 0x47, 2)
    return struct.unpack('>h', raw)[0] / 131.0

def calibrate_gyro():
    print("Calibrating Gyro... KEEP ROVER STILL.")
    bias = sum(read_gyro_z() for _ in range(100)) / 100.0
    print(f"Gyro Bias: {bias:.2f}")
    return bias


# ═══════════════════════════════════════════════════════
#  GYRO-CONTROLLED TURN
# ═══════════════════════════════════════════════════════
def gyro_turn(target_degrees, direction, gyro_bias):
    accumulated_angle = 0.0
    last_time = time.ticks_us()
    set_turn_motors(direction)
    L_ENA.duty_u16(SPEED_REGULAR)
    R_ENB.duty_u16(SPEED_REGULAR)
    while accumulated_angle < target_degrees:
        now = time.ticks_us()
```

```
        dt  = time.ticks_diff(now, last_time) / 1_000_000.0
        last_time = now
        gz = read_gyro_z() - gyro_bias
        accumulated_angle += abs(gz) * dt
        time.sleep_ms(5)
    stop()
    print(f"Turn complete: {accumulated_angle:.1f}°")



# ═══════════════════════════════════════════════════════════
#  LORA / UART COMMS
# ═══════════════════════════════════════════════════════════
def send_lora(payload: str):
    """Send a validated <MSG|R2|...> frame to ESP8266 over UART1."""
    frame = payload.strip()
    if not (frame.startswith("<") and frame.endswith(">")):
        return
    uart_lora.write((frame + "\n").encode())


def send_mpu_telemetry(gyro_bias):
    """Build and send an MPU-6050 telemetry packet."""
    ax, ay, az = read_accel()
    gz = read_gyro_z() - gyro_bias
    frame = (
        f"<MSG|{ROVER_ID}|DIR:{_current_motion}"
        f"|AX:{ax:.3f}|AY:{ay:.3f}|AZ:{az:.3f}"
        f"|GZ:{gz:.2f}>"
    )
```

```
    send_lora(frame)
    print("[MPU TX]", frame)


# ═══════════════════════════════════════════════════════════
#  BOOT SEQUENCE
# ═══════════════════════════════════════════════════════════
stop()
print("=" * 40)
print("ROVER 2 — MOVEMENT + LORA TELEMETRY")
print("=" * 40)

time.sleep(1)
gyro_z_bias = calibrate_gyro()
base_ax, base_ay, _ = read_accel()

print("\n>>> ROVER 2 ACTIVE <<<\n")


# ═══════════════════════════════════════════════════════════
#  MAIN LOOP
# ═══════════════════════════════════════════════════════════
try:
    while True:
        now_ms = time.ticks_ms()

        # ── 1. Periodic MPU telemetry ──────────────────────
        if time.ticks_diff(now_ms, _last_mpu_tx) >= MPU_TX_INTERVAL:
```

```
    send_mpu_telemetry(gyro_z_bias)
    _last_mpu_tx = now_ms

# ── 2. Read ultrasonics ──────────────────────────
dist_L = get_distance(UL_TRIG, UL_ECHO); time.sleep_ms(15)
dist_C = get_distance(UC_TRIG, UC_ECHO); time.sleep_ms(15)
dist_R = get_distance(UR_TRIG, UR_ECHO)

# ── 3. Accelerometer bump detection ──────────────
ax, ay, _ = read_accel()
bump_detected = (abs(ax - base_ax) > 0.4 or abs(ay - base_ay) > 0.4)

closest_dist = min(dist_L, dist_C, dist_R)

# ── 4. Avoidance logic ───────────────────────────
if bump_detected:
    stop()
    print("[IMPACT] Blind-spot hit! Reversing.")
    send_lora(f"<MSG|{ROVER_ID}|EVENT:IMPACT|BLIND_SPOT>")
    backward(SPEED_REGULAR); time.sleep_ms(1000)
    stop(); time.sleep_ms(200)

    gyro_turn(30, "right", gyro_z_bias); time.sleep_ms(100)
    scan_R = get_distance(UC_TRIG, UC_ECHO)

    gyro_turn(60, "left", gyro_z_bias); time.sleep_ms(100)
    scan_L = get_distance(UC_TRIG, UC_ECHO)
```

```
    if scan_R > scan_L:
        gyro_turn(60, "right", gyro_z_bias)

    time.sleep_ms(200)
    base_ax, base_ay, _ = read_accel()

elif closest_dist <= DIST_CRITICAL:
    stop()
    print(f"[CRITICAL] {closest_dist:.1f}cm — Emergency reverse.")
    send_lora(f"<MSG|{ROVER_ID}|EVENT:CRITICAL|DIST:{closest_dist:.1f}cm>")
    backward(SPEED_REGULAR); time.sleep_ms(1000)
    stop(); time.sleep_ms(200)

    print("[SCAN] Rotating to find clear path...")
    turn_left()
    while True:
        scan_C = get_distance(UC_TRIG, UC_ECHO)
        if scan_C > 45:
            print(f"[SCAN] Clear path {scan_C:.1f}cm — resuming.")
            break
        time.sleep_ms(40)

    stop(); time.sleep_ms(200)
    base_ax, base_ay, _ = read_accel()

elif closest_dist <= DIST_STOP:
    stop()
    print(f"[STOP] {closest_dist:.1f}cm — Evading.")
```

```
            send_lora(f"<MSG|{ROVER_ID}|EVENT:EVADE|DIST:{closest_dist:.1f}cm>")
            time.sleep_ms(200)
            if dist_L > dist_R:
                gyro_turn(30, "left", gyro_z_bias)
            else:
                gyro_turn(30, "right", gyro_z_bias)
            time.sleep_ms(100)
            base_ax, base_ay, _ = read_accel()

        elif closest_dist <= DIST_SLOW:
            print(f"[SLOW] {closest_dist:.1f}cm — Braking.")
            forward(SPEED_SLOW)

        else:
            forward(SPEED_REGULAR)

        time.sleep_ms(30)

except KeyboardInterrupt:
    stop()
    print("\n" + "=" * 40)
    print("EMERGENCY STOP TRIGGERED.")
    print("=" * 40)
```